

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y. 2026-27 [ODD]

LAB MANUAL

CEUC203: Object Oriented Programming

PRACTICAL-1

PART A

1. Vending machine (VendingMachine.java, main in this file)
   (a) Define an enum Coin with constants ONE, TWO, FIVE, TEN.
   (b) In main, set a snack price of 15 and a running total of 0; create a Scanner. (c) Loop: read a coin name, use a switch expression to convert the Coin to its value (ONE→1, TWO→2, FIVE→5, TEN→10), add it to the total, and print the total so far.
   (d) Stop the loop once the total reaches 15 or more.
   (e) Print the change to return (total − 15).
   Expected output: TEN then FIVE prints "Paid. Change: 0"; TEN then TEN prints "Paid. Change: 5"

CODE

```
import java.util.Scanner;
enum Coin
{
    ONE, TWO, FIVE, TEN
}
public class VendingMachine
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in); // to create an obj
        int price = 15; //snack price
        int total = 0; //money  inserted so far
        //keep asking until enough money is inserted
        while(total < price)
        {
            System.out.print("Enter coin (ONE, TWO, FIVE, TEN): ");
            String coin = sc.next();
            coin = coin.toUpperCase();
            Coin c = Coin.valueOf(coin);
            int value = 0;
            switch(c)
            {
                case ONE:
                    value = 1;
                    break;

                case TWO:
                    value = 2;
                    break;
```

```
                case FIVE:
                    value = 5;
                    break;

                case TEN:
                    value = 10;
                    break;
            }
            total = total + value;
            System.out.println("Total = " + total);
        }
        System.out.println("Paid.");
        System.out.println("Change = " + (total - price));
        sc.close();
    }
}
```

OUTPUT

```
PS C:\Users\diyat\OneDrive\Documents\Dia study\java lab>  & 'C:\Program Files (x86)\Java\jdk1.8.0_77\bin\java.exe' '-cp' 'C:\Users\diyat\AppData\Roaming\Code\User\workspaceStorage\12f178237875f72cef1cda6c5b724920\redhat.java\jdt_ws\java lab_7fa1aad4\bin' 'VendingMachine'
Enter coin (ONE, TWO, FIVE, TEN): TEN
Total = 10
Enter coin (ONE, TWO, FIVE, TEN): FIVE
Total = 15
Paid.
Change = 0
PS C:\Users\diyat\OneDrive\Documents\Dia study\java lab>
```

CONCLUSION

The Vending Machine program accepts coins from the user, calculates the total amount inserted, stops when the snack price is reached, and displays the change to be returned. It demonstrates the use of enum, switch-case, loops, and user input in Java.

2. Toll booth (TollBooth.java, main in this file)
   (a) Define a record Vehicle(String number, String type).
   (b) In main, keep a running toll total and three counters (bike, car, truck). (c) Loop until the user types "done" for the number: build a Vehicle, use a switch expression on its type for the toll (bike→20, car→50, truck→150), add to the total, and increment that type's counter.
   (d) After the loop, print the total toll and the type with the highest count.

Expected output: two cars + one bike → "Total toll: 120", "Most frequent: car"

CODE

```
import java.util.Scanner;

public class TollBooth
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        int totalToll = 0;
        int bikeCount = 0;
        int carCount = 0;
        int truckCount = 0;

        while (true)
        {
            System.out.print("Enter Vehicle Number (or done): ");
            String number = sc.next();

            if (number.equalsIgnoreCase("done"))
                break;

            System.out.print("Enter Vehicle Type (bike/car/truck): ");
            String type = sc.next().toLowerCase();

            int toll = 0;

            switch(type)
            {
                case "bike":
                    toll = 20;
                    bikeCount++;
                    break;

                case "car":
                    toll = 50;
                    carCount++;
                    break;

                case "truck":
                    toll = 150;
                    truckCount++;
                    break;
```

```
                default:
                    System.out.println("Invalid Vehicle Type");
                    continue;
            }

            totalToll += toll;
        }

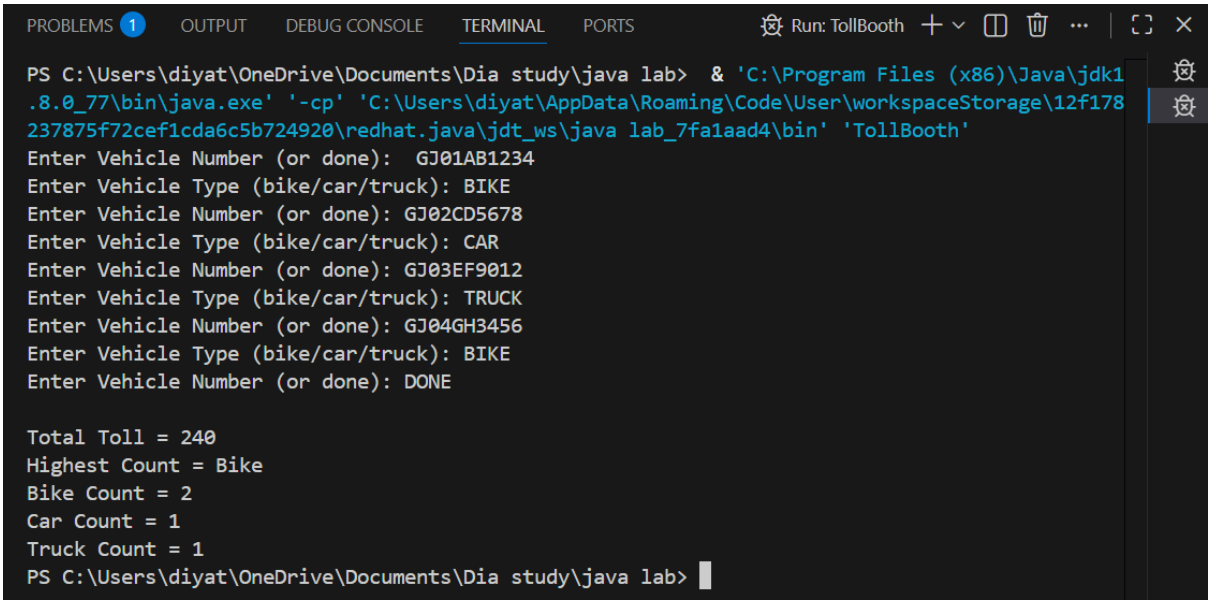
        System.out.println("\nTotal Toll = " + totalToll);

        if (bikeCount >= carCount && bikeCount >= truckCount)
            System.out.println("Highest Count = Bike");
        else if (carCount >= bikeCount && carCount >= truckCount)
            System.out.println("Highest Count = Car");
        else
            System.out.println("Highest Count = Truck");

        System.out.println("Bike Count = " + bikeCount);
        System.out.println("Car Count = " + carCount);
        System.out.println("Truck Count = " + truckCount);

        sc.close();
    }
}
```

OUTPUT



CONCLUSION

The Toll Booth program calculates the total toll collected based on the type of vehicle, counts the number of bikes, cars, and trucks, and displays which vehicle type has the highest count. It demonstrates the use of loops, switch-case, conditional statements, and user input in Java.

3. (For Advanced Learners – home) Rock-Paper-Scissors-Lizard-Spock (RPSLS.java)

(a)Define an enum Move with ROCK, PAPER, SCISSORS, LIZARD, SPOCK.

(b) Write winner(Move a, Move b) returning 1 if a beats b, -1 if b beats a, 0 for a tie — use a switch expression listing what each move beats (Scissors cut Paper, Paper covers Rock, Rock crushes Lizard, Lizard poisons Spock, Spock smashes Scissors, and the five mirror cases).

(c) In main, play 5 rounds: pick a random computer Move, read the player's Move, call winner(), print the round result and keep score.

(d) After 5 rounds, print the overall winner.

Expected output: each round prints both moves and the round winner; ends with a "You win 3–2" summary.

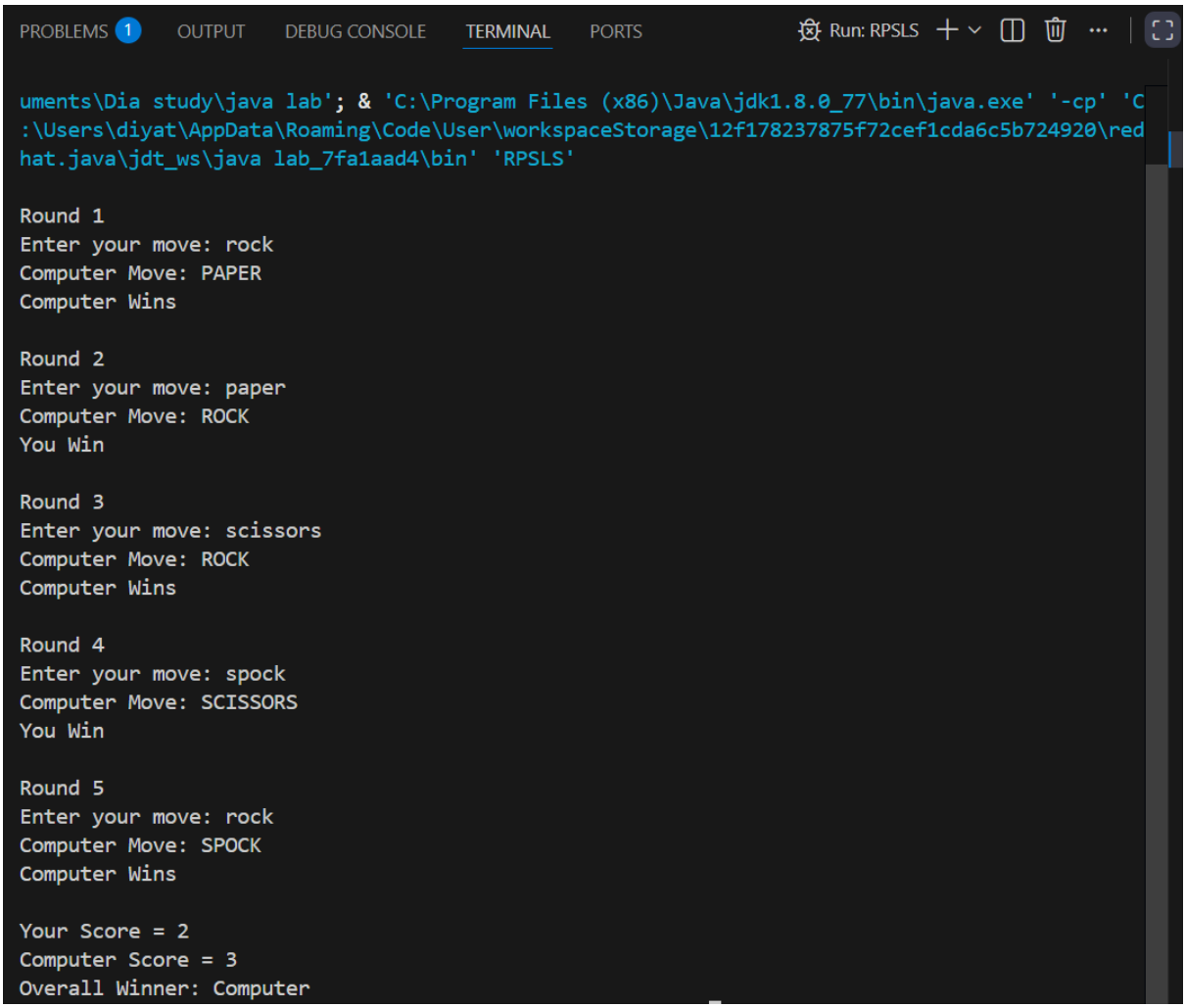
CODE

```
import java.util.Scanner;
enum Move
{
    ROCK, PAPER, SCISSORS, LIZARD, SPOCK
}
public class RPSLS
{
    static int winner(Move a, Move b)
    {
        if(a == b)
            return 0;
        if((a == Move.ROCK && (b == Move.SCISSORS || b == Move.LIZARD)) ||
           (a == Move.PAPER && (b == Move.ROCK || b == Move.SPOCK)) ||
           (a == Move.SCISSORS && (b == Move.PAPER || b == Move.LIZARD)) ||
           (a == Move.LIZARD && (b == Move.PAPER || b == Move.SPOCK)) ||
           (a == Move.SPOCK && (b == Move.ROCK || b == Move.SCISSORS)))
            return 1;

        return -1;
    }
    public static void main(String[] args)
    {
```

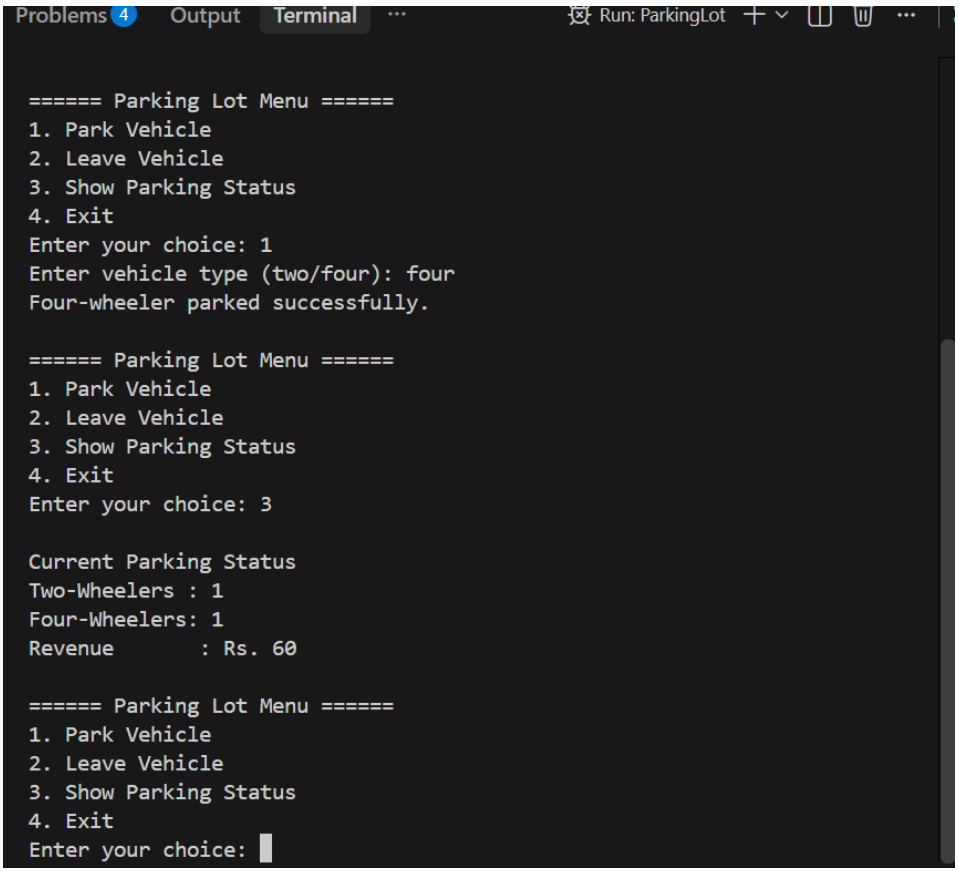
```
        Scanner sc = new Scanner(System.in);
        int user = 0;
        int comp = 0;
        for(int i = 1; i <= 5; i++)
        {
            System.out.println("\nRound " + i);
            System.out.print("Enter your move: ");
            Move player = Move.valueOf(sc.next().toUpperCase());
            int random = (int)(Math.random() * 5);
            Move computer;
            if(random == 0)
                computer = Move.ROCK;
            else if(random == 1)
                computer = Move.PAPER;
            else if(random == 2)
                computer = Move.SCISSORS;
            else if(random == 3)
                computer = Move.LIZARD;
            else
                computer = Move.SPOCK;
            System.out.println("Computer Move: " + computer);
            int result = winner(player, computer);
            if(result == 1)
            {
                System.out.println("You Win");
                user++;
            }
            else if(result == -1)
            {
                System.out.println("Computer Wins");
                comp++;
            }
            else
            {
                System.out.println("Tie");
            }
        }
        System.out.println("\nYour Score = " + user);
        System.out.println("Computer Score = " + comp);
        if(user > comp)
            System.out.println("Overall Winner: You");
        else if(comp > user)
            System.out.println("Overall Winner: Computer");
        else
            System.out.println("Overall Result: Match Tie");
        sc.close();
    }
}
```

OUTPUT



CONCLUSION

The RPSLS program compares the player's move with a randomly generated computer move, keeps the score for five rounds, and displays the overall winner. It demonstrates the use of enums, methods, loops, if-else statements, and random number generation in Java.



CONCLUSION

This practical helped me understand the use of classes, objects, constructors, private variables, final variables, and static variables in Java. It also showed how to manage parking operations, control vehicle capacity, and calculate total revenue using a simple menu-driven program.

KEY QUESTIONS

1. What is the role of the JVM, and what file does the javac compiler produce?

JVM (Java Virtual Machine):

- JVM executes Java bytecode.
- It makes Java platform independent ("Write Once, Run Anywhere").
- It manages memory, garbage collection, and security.

javac Compiler:

- The javac compiler converts a .java source file into a .class file.
- The .class file contains **bytecode**, which the JVM executes.

2. How does a switch expression differ from a traditional switch statement?

Traditional Switch Statement	**Switch Expression**
Uses case with :	Uses case with ->
Usually requires break	No break needed
Cannot directly return a value	Can directly return a value
More lines of code	Shorter and cleaner

3. Why is an enum a good choice for a fixed set of menu options, and a record for read-only data?

Enum

- Stores a fixed set of constants.
- Prevents invalid values.
- Improves code readability and type safety.

Record

- Used to store immutable (read-only) data.
- Automatically creates:
  - Constructor
  - Getters
  - toString()
  - equals()
  - hashCode()
- Reduces boilerplate code.